on this, we might want to keep track of three digital pins connected to a few LEDs or maybe an RGB LED. In that case, we need to declare the array and assign values to each index in the array, something like the following:

```
int ledPins[3];
ledPin[0] = 9;
ledPin[1] = 10;
ledPin[2] = 11;
```

This example works best in a situation where we need to set up the variable at the beginning of the sketch to set aside a chunk of memory, but we won't know what the values will be until later in the sketch. Maybe we are storing multiple sensor readings and need a temporary place to put them like we did with our project code. If, however, we already know what each element should contain, we could condense the four lines of code down to one, as in this following example:

```
int ledPins[] = {9, 10, 11};
```

Although functionally equivalent, there are two things different about this declaration from the previous example. First of all, this example is missing the index number inside the square brackets that tells us the dimension of the array. That's okay though because it is only necessary to indicate the size of the array when none of the values are being initialized. In this case because we have three values listed, the Arduino compiler figures out that there must be only three elements in the array and sets the dimension accordingly. It's still okay to throw the index in there, and it might be necessary if you only want to initialize a few of the values in a much larger array, but it is not always needed.

Secondly, the three values assigned to each element in the array are declared at the same time by enclosing them in curly braces, `{ }`, and separating each value with a comma. Because arrays are 0 indexed, the Arduino compiler assigns the first value in the list to index 0, the second to index 1, and so on until it has reached the end of the declared values. When all is said and done, `ledPin[0]` contains the value 9, `ledPin[1]` the value 10, and `ledPin[2]` the value 11. We should now take a moment to look at how to access arrays and how we can better use them in our sketches.

Using Arrays

By knowing how to properly declare an array and assign values to the individual elements, we can put the elements to use in a variety of different ways. To begin with, we can use the array just like any other variable except with an index number, as in this following example:

```
digitalWrite(ledPin[0], HIGH);
```

Assuming the previous array declaration with the value 9 assigned to the index of 0, this could be used to turn on an LED connected to I/O pin 9. One thing to mention is that it is possible to refer to an array without the square brackets because the name of the array points to the first element in the array. In that way, both `*ledPin` and `ledPin[0]` refer to the same value. Likewise, `ledPin[1]` is also the same as `*ledPin+1`. This little trick with the "*" symbol makes the array name into a pointer so that the compiler understands that this is a reference to an index position. Pointers are a more advanced topic that we can't really get into, but it's worth remembering because they could come in handy some time.

Anyway, instead of a number, we can also use a variable in the place of the array's index to access each element using something like a `for` loop. We have already snuck this in, in a previous example, with the following line of code:

```
for (int i=0; i<3; i++) digitalWrite(rgb[i], HIGH);
```